



河北大学

通信原理课程设计报告

题目：基于软件无线电平台和 FPGA
的 2ASK 调制解调系统设计

学 院
专 业
学 号
姓 名
指导教师

电子信息工程学院

2023 级通信工程 2 班

20231804086

柳晓阳

庞姣 田晓燕

2026 年 03 月 31 日

摘 要

本文针对现代数字通信系统中调制解调技术的应用与实践需求，基于 XSRP 软件无线电平台与 FPGA 硬件环境，设计并实现了一个完整的 2ASK（二进制振幅键控）调制解调系统。本设计采用自顶向下的模块化设计思路，使用 Verilog HDL 语言在 Quartus II 开发环境下，完成了 m 序列基带信号发生、正弦载波生成、乘法调制以及解调系统的核心代码编写与逻辑综合验证。

在解调部分，本系统不仅完成了基于全波整流、128 点滑动平均低通滤波和电平抽样判决的非相干解调基础任务，还创新性地设计并实现了一种高性能的相干解调方案。相干解调模块创新引入了极性 Costas 环结构，通过内部集成的 DDS、乘法鉴相器以及经过优化防积分饱和的环路滤波器，实现了本地恢复载波的精准同步提取，有效提升了解调精度。

最后，将程序编译下载至 XSRP 硬件平台，利用示波器对发射端调制波形、接收端各级处理（整流、滤波、判决）节点波形进行了全面观测与对比。软硬联调结果表明，本系统能够准确、稳定地完成 2ASK 信号的调制与解调，非相干与相干双路解调方案均能成功恢复出原始基带序列，圆满达到了课程设计的各项指标与创新要求。本设计不仅验证了《通信原理》的基础理论，也切实提升了软硬结合的工程实践能力。

关键词： 2ASK FPGA XSRP 软件无线电平台 相干解调 Costas 环

目 录

1 任务介绍	1
1.1 研究背景及意义	1
1.2 设计要求	1
2 方案设计	2
2.1 基本原理	2
2.1.1 2ASK调制原理	2
2.1.2 2ASK非相干解调原理	2
2.1.3 2ASK相干解调原理	3
2.2 方案论证	5
2.2.1 调制方案论证	5
2.2.2 解调方案论证	5
2.2.3 低通滤波方案论证	5
2.2.4 载波同步方案论证	6
3 设计与测试	6
3.1 各模块设计与测试	6
3.1.1 发射端及基础时钟模块设计	6
3.1.2 接收端：非相干解调模块设计	10
3.1.3 接收端：相干解调模块设计	13
3.2 整体测试	16
4 总结与展望	18
4.1 整体测试方案完成情况总结	18
4.2 系统局限性分析	19
4.3 能力提升与未来展望	19
5 参考文献	20
6 附录	21
6.1 顶层原理图	21
6.2 Verilog 代码	22

1 任务介绍

1.1 研究背景及意义

随着信息技术的飞速发展,软件无线电(SDR)技术与现场可编程逻辑门阵列(FPGA)在数字通信系统中的应用日益广泛。2ASK(二进制振幅键控)作为最基础的数字调制技术之一,其调制与解调过程涵盖了基带信号处理、载波同步、滤波与抽样判决等通信核心环节。深入研究2ASK系统的软硬件实现,不仅是理解复杂通信系统的基石,更具有广泛的工程应用背景:

光纤通信系统:在高速光纤通信中,2ASK的一种特殊形式——开关键控(OOK)是强度调制/直接检测(IM/DD)系统的核心技术,因其结构简单、成本低廉而在接入网中占据主导地位。

近场通信与物联网:在RFID标签、NFC支付及各类低功耗传感器节点中,2ASK调制因其对硬件处理能力要求较低、功耗极小,成为短距离低速无线传输的首选方案。

医疗电子与遥测系统:在植入式医疗设备或简单的工业遥测链路中,2ASK常用于传输关键的状态指令,确保在复杂的电磁环境下实现可靠的逻辑控制。

本设计的意义在于巩固《通信原理》中关于ASK调制解调的理论知识,并将理论应用到具体的工程实践中。通过在Quartus II软件环境下使用Verilog语言进行FPGA开发,掌握软硬件结合的通信系统构建流程。借助XSRP软件无线电平台进行系统验证,不仅能加深对通信理论的理解,还能切实提升解决实际工程问题的动手能力与创新思维。

1.2 设计要求

本系统具体的设计要求与指标如下:

功能要求:根据系统流程规划,理解并掌握各个程序模块的输入输出参数与功能原理,使用Verilog语言完成程序的编写。需实现的模块主要包括:系统分频模块、基于本原多项式产生伪随机序列的数字信号源模块、正弦载波生成模块、2ASK调制模块,以及接收端的整流模块、解调滤波模块和电平判决模块等。最终需将功能验证正常的各模块构成一个完整的系统,实现2ASK的调制与解调功能。

测试要求:系统需在XSRP软件无线电平台上运行,并利用示波器测量和观察各个关键测试点的波形。具体需对比观察的波形包括:载波信号与基带信号、调制后信号与基带信号、调制后信号与整流后信号、滤波前与滤波后信号,以及滤波后信号与最终的判决信号。

创新要求:在完成基础解调功能的前提下,编写接收端多种解调方法,如相干解调法。本设计需实现基于Costas环提取同步载波的相干解调方案,以提升系统的解调性能。

2 方案设计

2.1 基本原理

2.1.1 2ASK 调制原理

振幅键控（ASK）是指利用载波的振幅变化来传递数字基带信号的一种调制方式。在 2ASK 中，载波的振幅只有两种状态，分别对应数字基带信号的“0”和“1”，典型时域波形如图 1 所示：

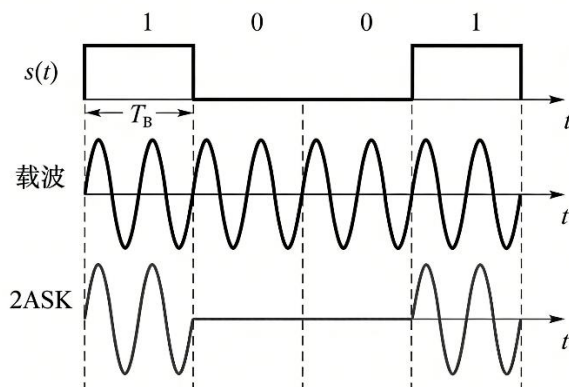


图1 2ASK调制时域波形

可见，2ASK 信号的包络完全随基带信号的规律变化，这种调制方式在时域上表现为载波的有无切换，其一般时域表达式为：

$$e_{2ASK}(t) = s(t)\cos(\omega_c t) \quad (1)$$

其中， $s(t)$ 为单极性非归零（NRZ）数字基带信号， $\cos(\omega_c t)$ 为高频正弦载波。

产生 2ASK 信号的方法通常有相乘法法和键控法两种，如图 2 所示：

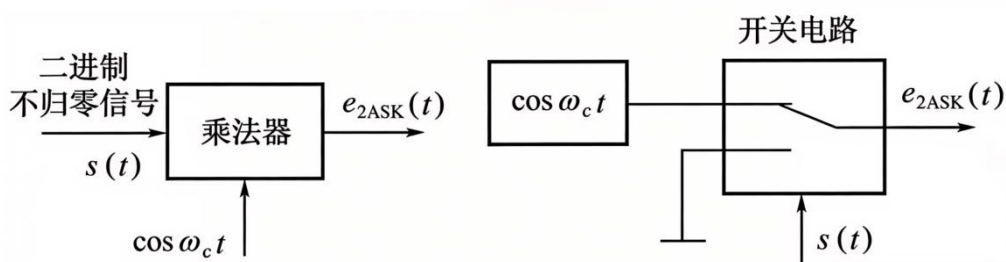


图2 两种2ASK调制方法

上图左是一般模拟幅度调制的方法，用乘法器实现；图右是数字键控法，其中的开关电路受 $s(t)$ 控制。

2.1.2 2ASK 非相干解调原理

2ASK 信号的非相干解调又称包络检波法，其核心优势在于接收端不需要提取与发送端同步的相干载波，系统结构简单且易于实现。本设计方案采用数字化的包络检波方式，其系统原理框图及各测试点波形如图 3、4 所示：

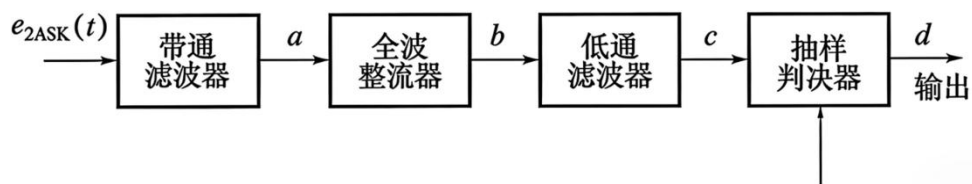


图3 2ASK 非相干解调原理框图

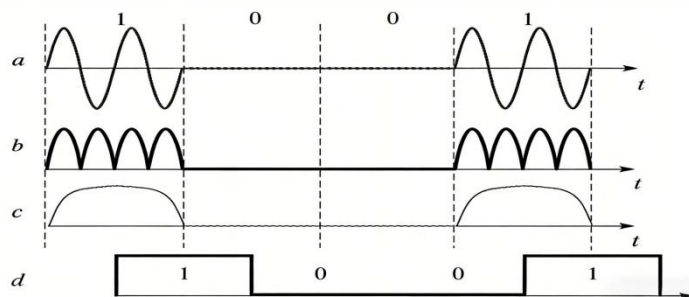


图4 2ASK 非相干解调各点波形对照图

结合上述图示，解调系统的具体实现过程如下：

带通滤波：已调信号首先经过带通滤波器，滤除信道中的带外噪声，得到较为纯净的已调波形。如波形 a 所示，在码元 1 期间载波存在，码元 0 期间载波为零。

全波整流：信号进入整流模块，对有符号数字信号取绝对值。如波形 b 所示，该过程将信号的负半轴波形翻转至正半轴，将交流载波转化为单向脉动直流信号，为提取包络做准备。

低通滤波：整流后的脉动信号送入低通滤波器。本设计通过 128 点滑动平均滤波算法，有效滤除高频纹波。如波形 c 所示，滤波器输出平滑的基带包络信号，真实反映了原始基带信号的变化趋势。

抽样判决：最后，包络信号进入判决模块。系统将包络电平与预设门限进行比较。如波形 d 所示，当电平高于门限时输出 1，低于门限时输出 0，从而成功恢复出原始二进制序列。

2.1.3 2ASK 相干解调原理

相干解调要求接收端能够产生一个与已调信号载波同频同相的本地参考载波。本设计采用 Costas 环法实现载波同步提取与信号解调，其原理框图如图 5 所示。

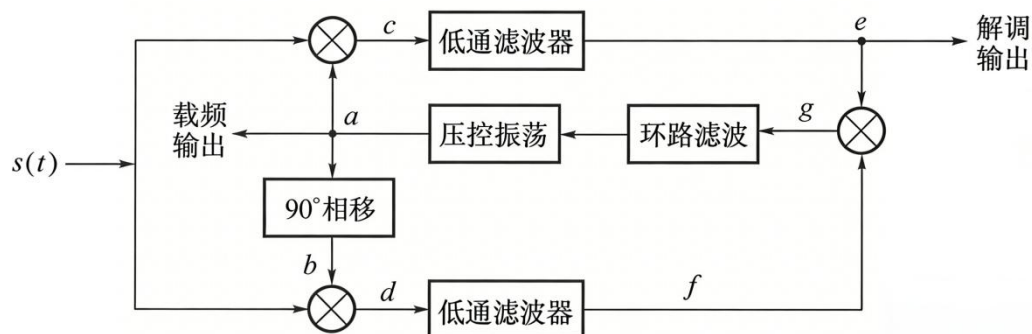


图5 Costa环法相干解调框图

假设输入的 2ASK 已调信号为：

$$s(t) = [A \cdot m(t)] \cos(\omega_c t + \theta) \quad (2)$$

其中， $m(t)$ 为单极性基带信号， ω_c 为载波角频率， θ 为待跟踪的初始相位。结合图 5 框图中的测试点，详细工作原理如下：

压控振荡器（VCO）输出本地同向载波：

$$a(t) = \cos(\omega_c t + \varphi) \quad (3)$$

其中 φ 为本地载波相位。该信号经过 90° 相移器后产生与之正交的本地载波：

$$b(t) = \sin(\omega_c t + \varphi) \quad (4)$$

输入信号 $s(t)$ 分别与两路本地载波在乘法器中相乘并利用积化和差公式得到 I 路、Q 路信号：

$$c(t) = s(t) \cdot a(t) = \frac{1}{2} A \cdot m(t) [\cos(\theta - \varphi) + \cos(2\omega_c t + \theta + \varphi)] \quad (5)$$

$$d(t) = s(t) \cdot b(t) = \frac{1}{2} A \cdot m(t) [\sin(\varphi - \theta) + \sin(2\omega_c t + \theta + \varphi)] \quad (6)$$

接着，两路信号分别通过同参数低通滤波器滤除 $2\omega_c$ 的高频分量，保留包含相位差信息的低频分量：

$$e(t) = \frac{1}{2} A \cdot m(t) \cos(\Delta\phi) \quad (7)$$

$$f(t) = \frac{1}{2} A \cdot m(t) \sin(\Delta\phi) \quad (8)$$

其中 $\Delta\phi = \varphi - \theta$ ，即本地载波与原始载波相位差。

将 I、Q 两路低通信号再次相乘，用于提取相位误差：

$$g(t) = e(t) \cdot f(t) = \frac{1}{8} A^2 \cdot m^2(t) \sin(2\Delta\phi) \quad (9)$$

由于 $m(t)$ 取值为 0 或 1，则 $m(t)^2$ 在有信号时恒为 1，这意味着误差信号的极性仅取决于相位差 $\Delta\phi$ ，不受调制信号的干扰。

误差信号 $g(t)$ 经过环路滤波器平滑处理后，输出控制电压作用于 VCO。该反馈机制会动态调整 VCO 的频率，促使本地载波相位 φ 向输入信号相位 θ 靠拢。当 $\Delta\phi$ 趋于 0 时， $e(t) = \frac{1}{2} A \cdot m(t)$ ， $f(t) \approx 0$ ，误差函数 $g(t) \approx 0$ ，环路进入锁定状态，此时点 e 输出即为稳定恢复的基带包络。

为了将该模拟信号还原为二进制比特流，系统需对点 e 的输出进行抽样判决。判决模块将包络信号的幅度与预设门限进行实时对比。若信号电平高于该门限，则判决输出为逻辑 1，反之判决为 0，最终实现原始数字基带序列的准确恢复。

2.2 方案论证

在基于 FPGA 的 2ASK 通信系统设计中, 针对调制、解调、滤波及载波同步等核心环节, 存在多种实现方案。结合任务书要求、FPGA 硬件平台特性及系统性能指标, 本节对各子系统的候选方案进行对比与论证。

2.2.1 调制方案论证

方案一: 键控法, 利用基带信号 $m(t)$ 作为数据选择器的控制端。当 $m(t) = 1$ 时输出载波, 当 $m(t) = 0$ 时输出 0。该方案仅需多路选择器, 几乎不消耗逻辑运算资源。

方案二: 相乘法法, 直接将带符号的载波信号与无符号的基带序列相乘。

方案选型: 考虑到现代 FPGA 内部拥有丰富的硬件乘法器资源, 且相乘法在代码结构上更为直观, 具有极强的移植性。同时, 相乘法也是高阶调制 (如 QAM) 的基础, 因此本设计选用方案二实现 2ASK 调制。

2.2.2 解调方案论证

方案一: 非相干解调 (包络检波法)。采用“全波整流 + 低通滤波 + 门限判决”的结构。其优点是无需载波同步, 结构极简, 系统延迟小, 占用 FPGA 逻辑资源极少; 缺点是抗加性高斯白噪声性能较差, 在低信噪比下易出现误码判决。

方案二: 相干解调 (同步检波法)。采用“本地同频同相载波混频 + 低通滤波 + 判决”的结构。其优点是抗干扰能力极强, 解调灵敏度高; 缺点是必须引入复杂的载波恢复环路, 消耗大量寄存器和乘法器资源, 且存在环路锁定时间。

方案选型: 为了保证基础功能的同时追求极致的系统性能, 本设计同时实现了方案一和方案二。在实际工程落地时, 非相干解调作为基础模块保障通信下限, 而相干解调作为创新模块拉高抗干扰上限, 二者完美支撑了任务的全面完成。

2.2.3 低通滤波方案论证

方案一: 传统 FIR/IIR 滤波器。借助 MATLAB 设计滤波器系数。IIR 存在相位非线性问题; 传统 FIR 则需要消耗大量的乘法器资源来进行卷积运算且设计难度较大。

方案二: 滑动平均滤波器, 这是一种特殊的 FIR 滤波器, 其所有抽头系数均为 1。由于不需要与不同权重的系数相乘, 它可以将复杂的乘累加运算简化为纯粹的加减法运算。此外, 当滤波器的阶数选取为 2 的整数次幂时, 最后的求平均除法运算可以直接通过移位或截取高位来实现, 极其节省硬件资源。

方案选型: 在非相干和相干解调中均需要提取低频基带信号, 本设计最终选用 128 点滑动平均滤波方案。该方案巧妙利用移位寄存器构建滑动窗口, 通过“累加新值, 减去最老值”的累加器结构更新总和, 最后通过算术右移代替乘法器实现除法。此方案完美契合 FPGA 擅长移位和加减运算的硬件特性, 在极低资源占用下实现了平滑的包络提取。

2.2.4 载波同步方案论证

方案一：平方环，对接收信号进行平方处理以消除调制信息，提取两倍频载波后再分频。此方案会引入平方损耗，恶化信噪比。

方案二：极性 Costas 环，利用本地 DDS 产生正交载波并进行混频，通过鉴相器提取误差并反馈控制。

方案选型：本设计选用极性 Costas 环方案。针对 2ASK 信号“0”码元无载波的特性，在 Costas 环中引入飞轮保持逻辑（当解调幅度过低时暂停积分更新），有效防止了环路滤波器积分爆炸，保证了 FPGA 闭环系统的绝对稳定。

3 设计与测试

3.1 各模块设计与测试

3.1.1 发射端及基础时钟模块设计

(1) 时钟分频模块

在整个 2ASK 调制解调系统中，各个子系统模块需要在不同的时钟频率下协同工作。本系统 XSRP 硬件平台提供的系统工作时钟为 26MHz。为了满足后续电路的时序要求，需设计一个通用的、支持占空比可调的任意整数分频器模块，以产生载波生成所需的 100KHz 时钟、M 序列生成所需的 25KHz 时钟，以及用于相干解调环路滤波器采样的 500KHz 时钟。

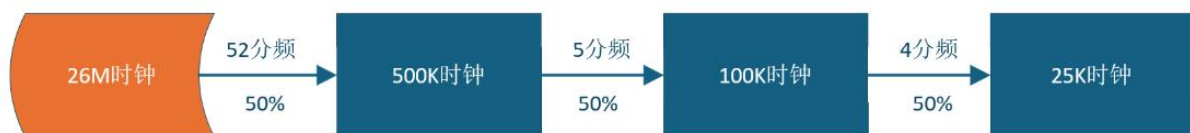


图6 时钟模块设计流图

由图 6 所示，本设计的时钟分频模块采用计数器比较法实现灵活可控的时钟分频功能，模块集成系统时钟 `clk_in`、分频系数 `div_ratio`、占空比控制参数 `duty_cycle` 三个输入端口；模块内部以 16 位计数器 `cnt` 为核心，在系统时钟上升沿驱动下完成累加计数，当计数值达到 `div_ratio-1` 时自动清零，以此确定输出时钟的完整周期，同时通过条件判断实现占空比精准控制，当计数器值小于占空比参数时输出时钟 `clk_out` 置高电平，反之置低电平，最终实现周期与占空比均可调的稳定时钟输出。

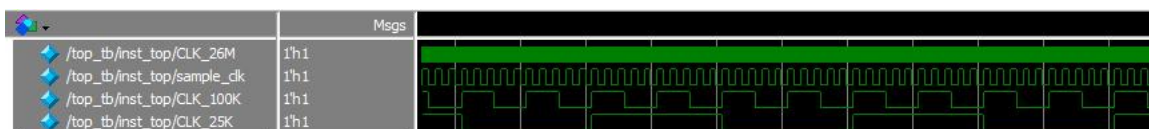


图7 时钟模块仿真图

由图 7 所示，系统所需 500KHz、100KHz、25KHz 时钟均能精确产生。

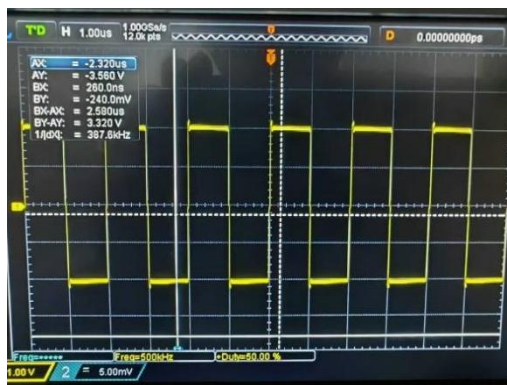


图8 500KHz时钟实测图



图9 100KHz时钟实测图

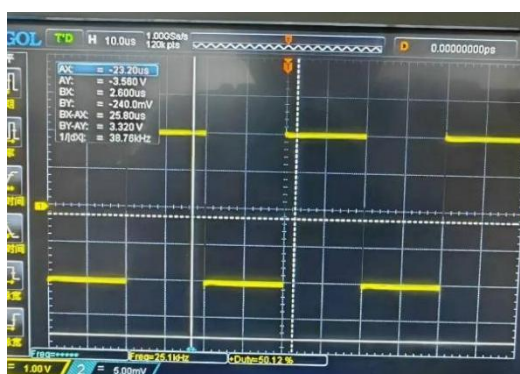


图10 25KHz时钟实测图

由图 8、图 9 和图 10 所示，三类时钟均为标准方波，占空比 50%、误差较小，无毛刺与相位偏移，完全匹配系统的时序需求。

(2) m 序列发生模块

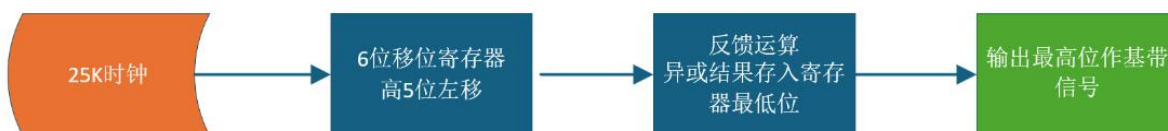


图11 m序列发生模块设计流程图

由图 11 所示，m 序列发生模块基于 6 级线性反馈移位寄存器设计，是系统基带信号的核心生成单元，专为 2ASK 调制提供周期固定、特性优良的伪随机基带信号。模块

以 25KHz 时钟为驱动信号，内部配置 6 位移位寄存器 m_reg ，初始状态设为全 1（避免全零状态导致寄存器死锁）；采用本原多项式 $X^6 + X^5 + 1$ 构建反馈逻辑，在时钟上升沿触发移位操作：将寄存器低 5 位 $m_reg[4:0]$ 左移作为新的高 5 位，同时将寄存器最高位 $m_reg[5]$ 与次高位 $m_reg[4]$ 进行异或运算，运算结果作为反馈信号输入寄存器最低位；最终提取移位寄存器的最高位 $m_reg[5]$ 作为输出基带信号 m_seq ，生成周期为 63 的标准 6 级 m 序列，为后续调制模块提供稳定的单极性 NRZ 伪随机基带信号。

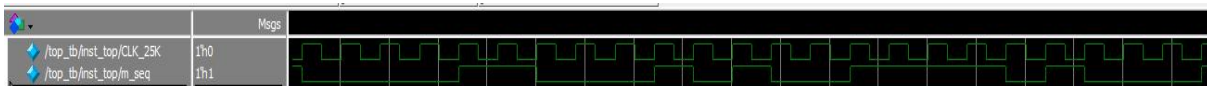


图12 m序列发生模块仿真图

由图 12 所示， m 序列发生模块的 Modelsim 功能仿真波形显示，输出 m_seq 在 25KHz 驱动时钟 CLK_25K 的上升沿同步更新，码元宽度严格等于时钟周期（40 μ s），码元速率为 25kbps，未出现全零死锁状态，符合 6 级 LFSR 本原多项式的反馈逻辑，波形无毛刺与亚稳态，验证了模块时序逻辑的严谨性与功能正确性。

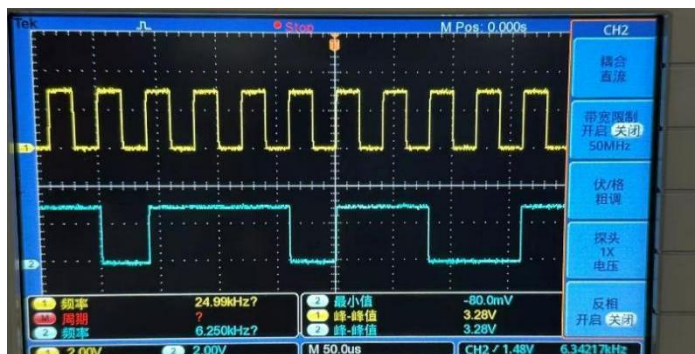


图13 m序列发生模块实测图

由图 13 所示，示波器实测波形中，CH1 时钟信号频率为 24.99kHz，与设计值 25kHz 误差仅 0.04%，CH2 m 序列信号峰峰值 3.28V，电平接近 FPGA 3.3V 逻辑标准，且电平跳变严格同步于时钟上升沿；序列高低电平随机分布，波形形态与仿真结果高度一致，验证了硬件实现的时序正确性与伪随机特性，微小频率与电平偏差属于硬件正常噪声，不影响模块功能。

(3) 原始载波发生模块



图14 原始载波发生模块设计流图

由图 14 所示，载波生成模块采用 ROM 查表法实现高精度正弦载波输出，以 26MHz 系统时钟为驱动，内部集成 9 位地址计数器与 260 深度的 Block ROM 存储单元。计数器在时钟上升沿循环累加，地址范围 0~259，循环读取 ROM 中预存的 8 位无符号正弦波量化数据；通过将读取数据减去 127，完成无符号数到有符号数的格式转换，最终输

出平滑稳定的 8 位有符号正弦载波信号。模块强制使用 FPGA Block RAM 资源，保障高速时钟下的数据读取稳定性，为系统调制环节提供符合要求的载波信号。

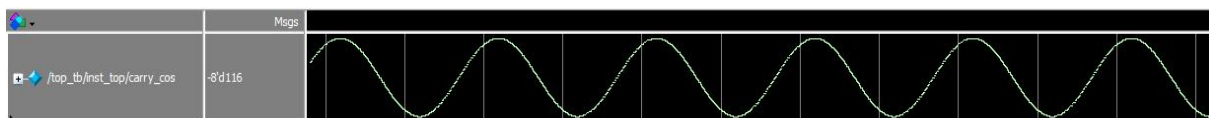


图15 原始载波发生模块仿真图

由图 15 功能仿真波形显示，输出 `carry_cos` 为平滑连续的 8 位有符号正弦波，峰峰值覆盖 ± 127 ，与 ROM 预存正弦量化数据的转换逻辑完全一致；在 26MHz 时钟驱动下，260 点采样周期对应输出频率为 100kHz，验证了 ROM 查表法载波生成的频率准确性与波形完整性，无失真、毛刺或相位偏移。

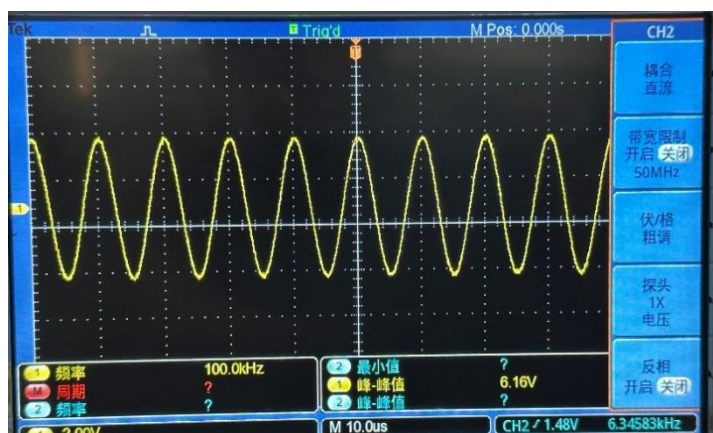


图16 原始载波发生模块实测图

由图 16 所示，示波器实测波形为标准正弦信号，频率精确为 100.0kHz，与设计目标完全匹配；峰峰值 6.16V，波形平滑无杂波、过冲或失真，验证了硬件实现的载波信号完整性与频率稳定性，为后续 2ASK 调制环节提供了高质量的载波基准。

(4) 调制模块

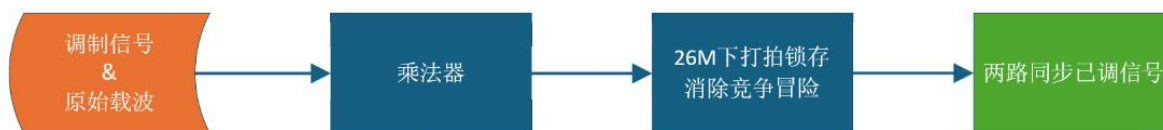


图17 调制模块设计流程图

由图 17 所示，本模块采用数字相乘法实现 2ASK 幅度键控，是发射端的核心调制单元。以单极性 m 序列基带信号为开关控制信号，与输入正弦载波直接进行数字乘法运算：基带信号为 1 时输出原始载波，为 0 时输出零电平，完成 2ASK 信号调制。为消除组合逻辑产生的竞争冒险，在 26MHz 时钟上升沿对乘法结果进行寄存器打拍锁存，最终输出两路完全一致的稳定已调信号，分别供给相干解调与非相干解调模块使用。

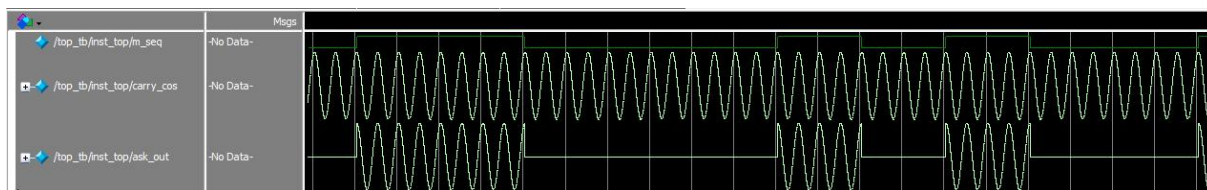


图18 调制模块仿真图

由图 18 所示，调制模块仿真波形清晰呈现 2ASK 信号特征，输出信号随基带 m_seq 同步跳变，基带为 1 时输出完整正弦载波，为 0 时保持零电平，寄存器打拍处理有效消除毛刺与干扰，波形规整无失真，验证 2ASK 调制逻辑的正确性。

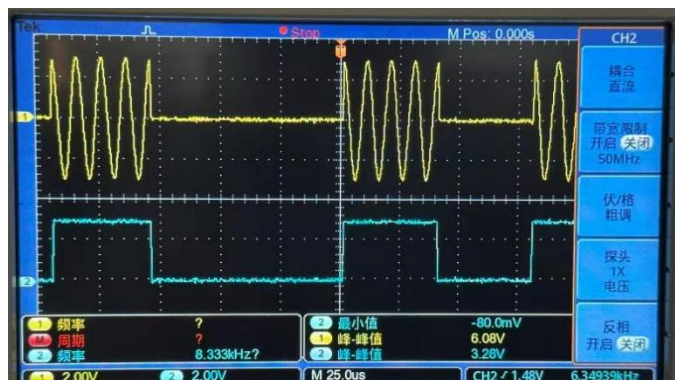


图19 调制模块实测图

由图 19 所示，硬件实测 2ASK 已调信号包络严格跟随基带信号变化，“1”码元对应平滑正弦载波，“0”码元为零电平，信号无畸变、同步性良好，与仿真波形高度吻合，证明调制模块硬件实现稳定可靠，满足系统调制要求。

3.1.2 接收端：非相干解调模块设计

(1) 全波整流模块

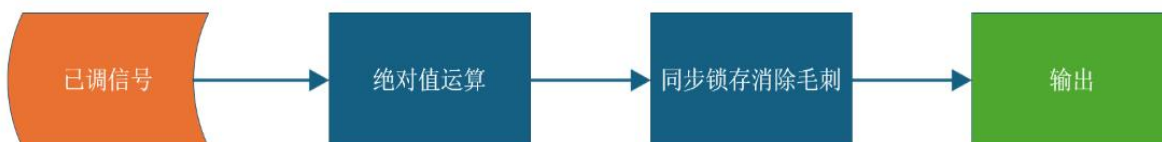


图20 全波整流模块设计流图

由图 20 所示，本模块为非相干解调的核心预处理单元，采用全波整流法提取 2ASK 信号包络：通过组合逻辑对输入有符号已调信号取绝对值（负数通过补码反相加一转换为正电平，正数直接输出），将双向交流载波信号转换为单向脉动直流信号；为避免组合逻辑毛刺，在时钟上升沿对整流结果打拍锁存，输出稳定的无符号信号，为后续低通滤波提取平滑基带包络提供可靠输入，保障非相干解调的包络提取精度。

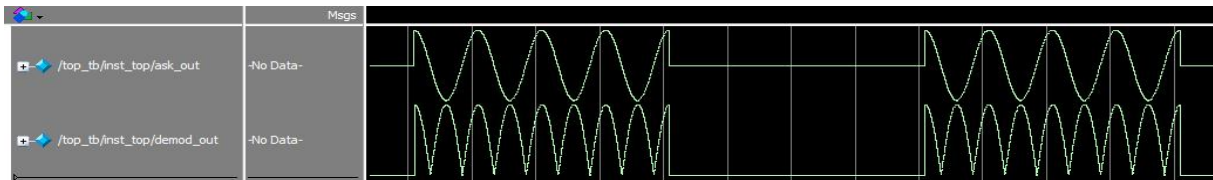


图21 全波整流模块仿真图

由图 21 所示，仿真波形清晰展示了全波整流模块的包络提取效果：输入 ask_out 为双向 2ASK 正弦信号，经整流后输出 demod_out 转换为单向脉动直流信号，负半周被完整翻转至正半周，包络严格跟随原 2ASK 信号的高低电平变化，与基带 m 序列特征完全匹配；时钟同步锁存有效消除了组合逻辑毛刺，整流后波形平滑无失真，验证了模块能精准提取 2ASK 信号包络，为后续低通滤波提供稳定输入。

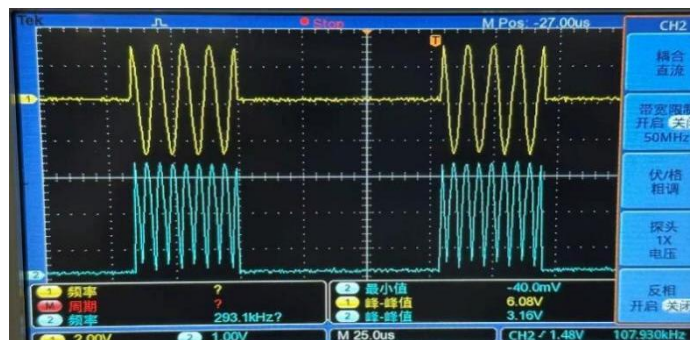


图22 全波整流模块实测图

由图 22 所示，示波器实测波形中，CH1 为 2ASK 已调信号，CH2 为整流输出信号：整流后信号无负电平，呈现单向脉动形态，峰峰值 3.16V，包络与 CH1 的 2ASK 信号完全同步，“1”码元对应高频脉动直流，“0”码元保持零电平，信号无畸变、无杂波，与仿真波形高度吻合，证明硬件全波整流模块功能稳定可靠，成功完成 2ASK 信号的包络预处理。

(2) 128 点滑动平均滤波模块

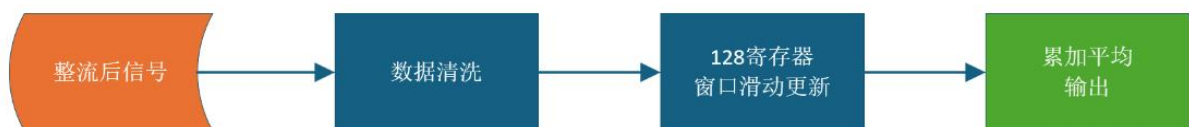


图23 128点滑动平均滤波模块设计流图

由图 23 所示，本模块是非相干解调的核心滤波单元，采用 128 级 8 位移位寄存器构建滑动窗口，实时存储历史采样数据；通过增量式累加器替代传统乘累加运算，仅用加减法完成窗口内数据求和（新数据加入、最老数据移出），大幅节约硬件资源；新增不定态清洗逻辑，将输入 X 态数据置 0，避免异常值干扰滤波效果；最终通过算术右移 7 位等效实现除以 128 的平均运算，滤除 2ASK 信号中的载波高频纹波，输出平滑的基带包络信号。该结构无乘法器、资源占用低、运算速度快，完美适配 FPGA 高速数字信号处理场景。

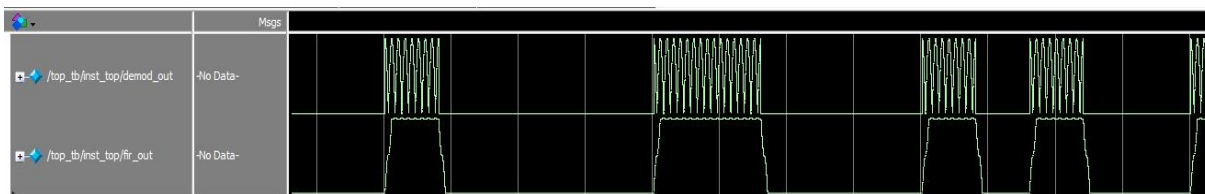


图24 128点滑动平均滤波模块仿真图

由图 24 所示，仿真波形中，输入 `demod_out` 为高频脉动的整流后信号，经 128 点滑动平均滤波后，输出成功滤除 200kHz 高频纹波，得到平滑的基带包络信号，包络高低电平与原 `m` 序列基带信号完全对应；滤波后波形存在轻微窗口延迟，但不影响后续判决，验证了模块的低通滤波效果，可有效提取 2ASK 信号的平滑包络，为判决模块提供可靠输入。



图25 128点滑动平均滤波模块实测图

由图 25 所示，示波器实测波形中，CH2 为整流后的高频脉动信号，CH1 为滤波输出信号：模块成功滤除载波高频纹波，输出平滑的基带包络，高低电平分界清晰，包络形态与仿真结果高度一致，严格跟随原 2ASK 信号的基带变化，无明显失真与杂波，验证了硬件滤波模块的稳定性与有效性，为非相干解调的判决环节提供了高质量的平滑包络。

(3) 非相干解调抽样判决模块



图26 非相干解调抽样判决模块信号流图

由图 26 所示，抽样判决模块是非相干解调的最终环节，用于将滤波后的平滑包络信号还原为原始数字基带信号。模块以系统时钟为同步基准，在时钟上升沿对输入的滤

波包络信号 `data_in` 进行采样；预设判决门限为 `8'd40`（对应 2ASK 包络峰值的 1/2），通过比较器逻辑判断输入信号电平：当信号大于门限时输出高电平 1，小于等于门限时输出低电平 0；最终经寄存器锁存输出稳定的 `data_out`，完成基带 `m` 序列的恢复，实现 2ASK 信号的完整非相干解调。

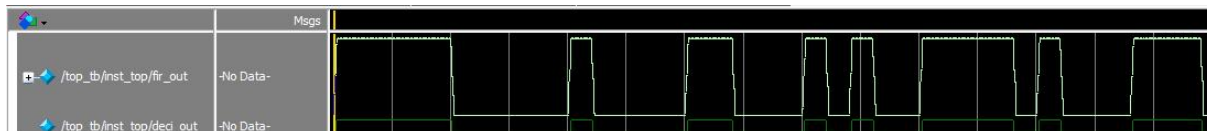


图27 非相干解调抽样判决模块仿真图

由图 27 所示，仿真波形清晰展示了抽样判决模块的解调效果：输入为平滑基带包络信号，经门限 `8'd40` 判决后，输出 `deci_out` 严格跟随包络电平变化，包络高于门限时输出高电平 1，低于门限时输出低电平 0，恢复的基带信号与原始 `m` 序列完全同步，无错码、无抖动，验证了门限设置的合理性与判决逻辑的正确性，成功完成 2ASK 信号的非相干解调。

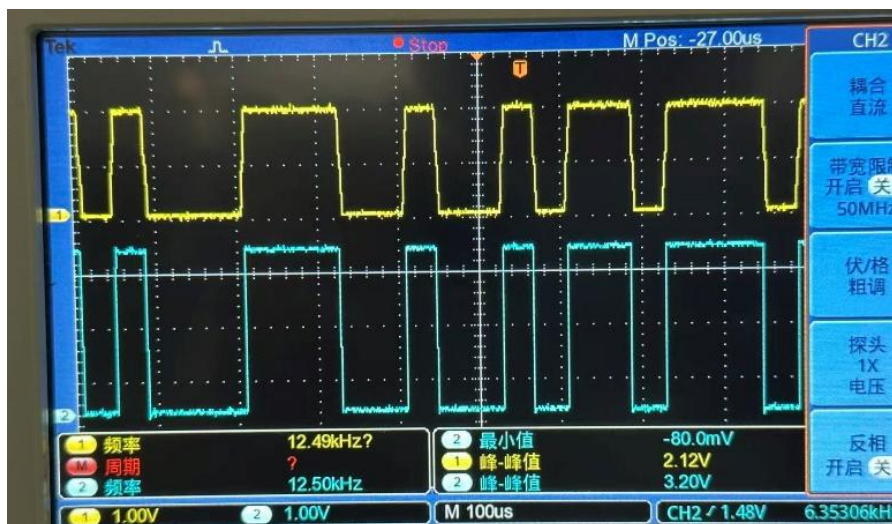


图28 非相干解调抽样判决模块实测图

由图 28 所示，示波器实测波形中，CH1 为滤波后平滑包络信号，CH2 为判决输出基带信号，输出方波与包络高低电平严格对应，电平跳变同步，无畸变、无误判，恢复的基带信号与发射端 `m` 序列特征一致，验证了硬件抽样判决模块的稳定性与解调可靠性，完成非相干解调的最终信号还原。

3.1.3 接收端：相干解调模块设计

(1) DDS 本地载波生成模块

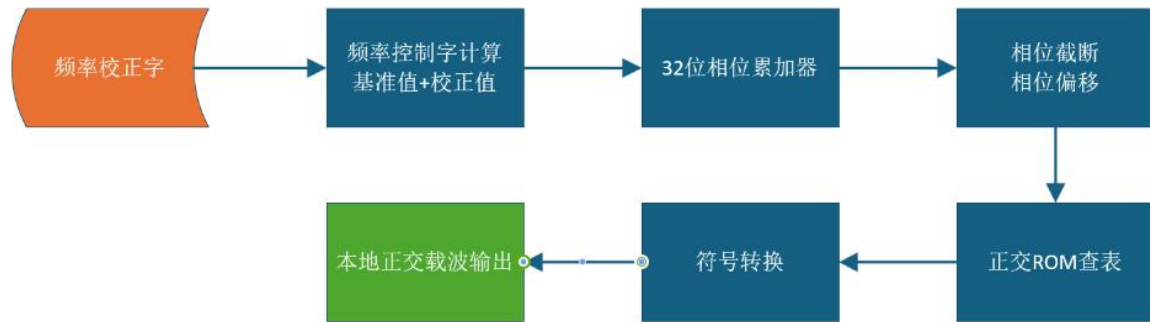


图29 DDS本地载波生成模块设计流图

由图 29 所示，本模块采用直接数字频率合成技术，为相干解调系统提供正交正弦/余弦载波，是 Costas 环同步的核心单元。模块以 32 位相位累加器为核心，通过固定基准频率控制字与外部 `freq_adj` 动态叠加，实现载波频率精准调节；相位累加器持续累加生成连续相位序列，截取高 8 位作为 ROM 地址，并叠加 45° 初始相位偏移，用于验证载波同步环路的锁定能力；通过两路独立 ROM 分别生成正弦、余弦信号，将 ROM 输出的无符号量化值减去 127 转换为 8 位有符号信号，最终输出一对正交、频率可控的稳定载波，为后续相干混频模块提供标准同步载波输入。

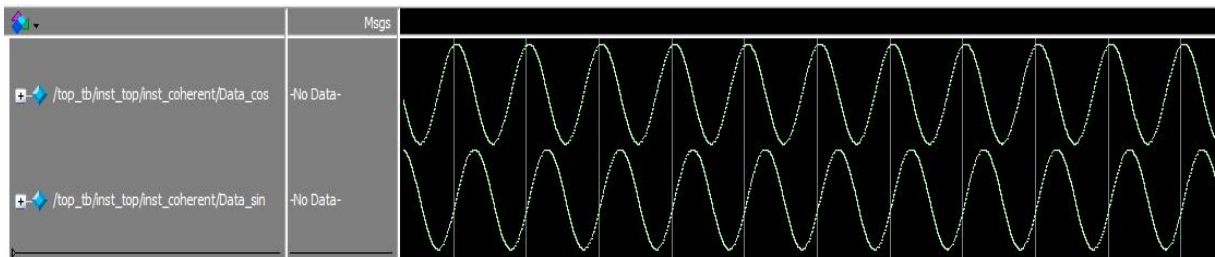


图30 DDS本地载波生成模块仿真图

由图 30 所示，仿真波形中，`Data_sin` 与 `Data_cos` 为一对平滑连续的 8 位有符号正弦 / 余弦信号，两者严格保持 90° 相位正交，频率稳定且无失真、毛刺，验证了 DDS 模块相位累加、ROM 查表及正交地址偏移逻辑的正确性，为后续相干解调提供了标准正交载波。

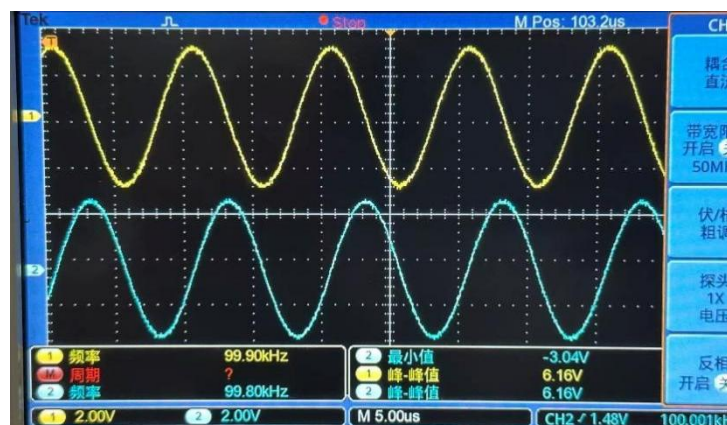


图31DDS本地载波生成模块实测图

由图 31 示波器实测波形显示，两路载波频率分别为 99.90kHz 与 99.80kHz，与设计

目标 100kHz 高度吻合，峰峰值均为 6.16V，波形平滑无杂波，正交相位关系清晰，验证了硬件 DDS 模块的频率精度与信号完整性，满足相干解调对本地同步载波的要求。

(2) Costas 环路滤波与鉴相反馈控制

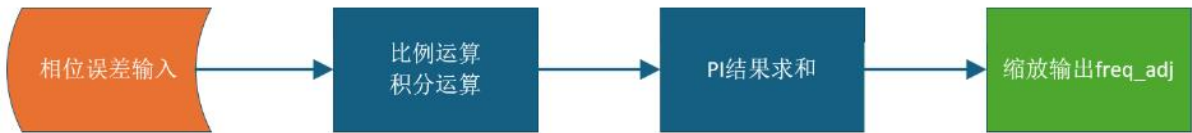


图32 Costas环路滤波与鉴相反馈控制模块设计流程图

由图 32 所示，本模块 Costas 载波同步环的核心反馈控制单元，采用带抗积分饱和和优化的比例 - 积分（PI）算法，实现相位误差的滤波与闭环调节。模块接收鉴相器输出的相位误差 `error_in`，通过比例系数 K_p 快速响应相位偏差、积分系数 K_i 消除稳态误差；`code_en` 为飞轮使能信号，有效时执行闭环控制，积分器带上下限幅保护，防止累加溢出导致环路失控；无效时积分器自动缓慢泄放归零，避免环路失锁后状态异常。模块将比例与积分运算结果求和后，通过算术右移 4 位完成信号缩放，最终输出频率调节信号 `freq_adj`，实时修正 DDS 本地载波参数，实现与接收信号的精准同频同相锁定，为相干解调提供稳定的同步载波。

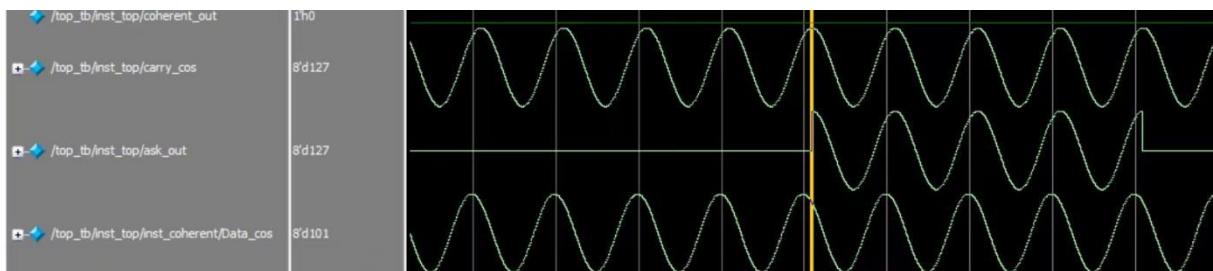


图33 Costas环锁定前仿真图

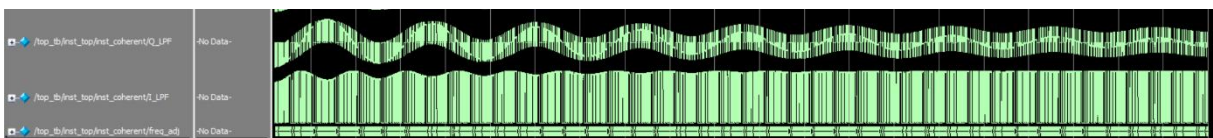


图34 Costas环锁定过程仿真图

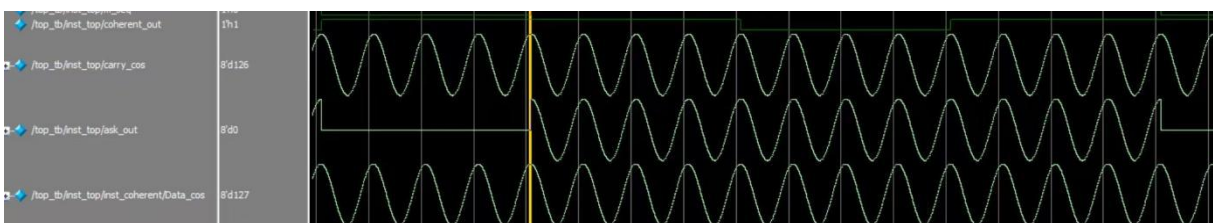


图35 Costas环锁定后仿真图

由图 33、图 34 和图 35 所示，仿真波形清晰展示了 Costas 环从失锁到锁定的完整过程：

锁定前，本地载波 `Data_cos` 与接收载波 `carry_cos` 存在明显相位差，I/Q 两路低通滤波输出 `I_LPF/Q_LPF` 幅值较大、相位波动明显，环路误差信号活跃，`freq_adj` 持续调

整以追踪频偏。

锁定过程，随着 PI 环路滤波与反馈调节，本地载波逐步向接收载波逼近， I_{LPF} 逐渐向 0 收敛， Q_{LPF} 幅值逐渐平稳，近乎常数，误差信号减小， $freq_adj$ 动态调整幅度降低。

锁定后，本地载波与接收载波实现同频同相， I_{LPF}/Q_{LPF} 稳定跟随 2ASK 包络变化， $freq_adj$ 维持在极小波动范围，最终解调输出 $coherent_out$ 准确还原原始基带信号，验证了环路锁定的有效性。

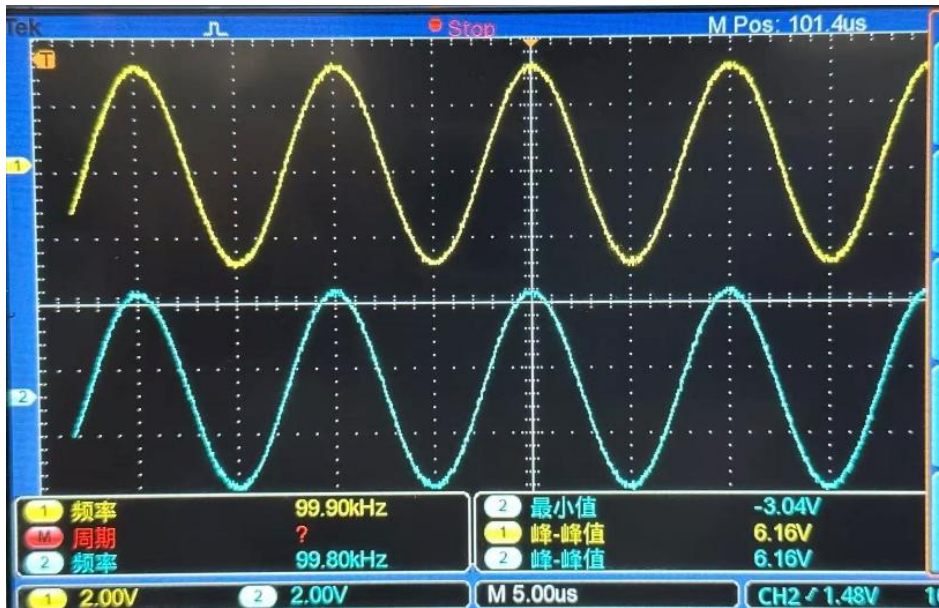


图36 Costas环锁定后实测图

由图 36 示波器实测波形显示，锁定后两路正交载波频率分别为 99.90kHz 与 99.80kHz，与设计目标 100kHz 高度一致，峰峰值均为 6.16V，波形平滑无杂波，相位正交关系清晰稳定；本地载波与接收载波实现精准同步，无频率漂移与相位偏移，为后续相干解调提供了可靠的同步基准，验证了硬件 Costas 环锁定性能稳定可靠。

(3) 混频、滤波与判决模块

相干解调在完成本地同步载波提取后，通过乘法混频将 2ASK 已调信号与本地同频同相载波相乘，得到包含基带信息的差频信号；后续的低通滤波、抽样判决处理流程与非相干解调模块的滤波、判决原理高度一致，均通过滑动平均滤波滤除高频分量、固定门限判决恢复基带信号，此处不再重复展开分析。

3.2 整体测试



图37 顶层原理图

由图 37 所示，本系统为基于 FPGA 的 2ASK 数字通信系统，采用非相干解调与相干解调双架构并行设计，整体模块化划分清晰，便于调试与扩展：

时钟模块（Clock module）：生成系统所需的 26MHz 主时钟及 25kHz 基带采样时钟，为全系统提供时序基准。

IIC 模块（IIC module）：负责外设配置与数据交互，实现系统参数的动态配置。

发射模块（Transmission module）：包含 m 序列生成单元、100kHz 正交载波生成单元（DDS）及 2ASK 调制单元，完成基带信号到射频已调信号的转换。

非相干解调模块（Noncoherent module）：由全波整流、128 点滑动平均滤波及抽样判决单元组成，无需同步载波即可完成信号解调。

相干解调模块（Coherent module）：包含 Costas 环载波同步单元、混频单元、滤波及判决单元，通过本地载波同步实现更高精度的解调。

观测模块（Observe module）：提供多路信号输出接口，便于示波器观测与调试，验证各模块功能。



图38 整体输入输出仿真图（非相干解调）

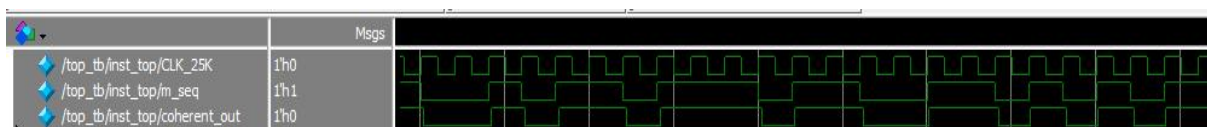


图39 整体输入输出仿真图（相干解调）

由图 38 和图 39 仿真测试结果显示，非相干解调链路中，输入 25kHz 基带 m 序列经 2ASK 调制、全波整流、滑动平均滤波与抽样判决后，输出解调信号与原始 m 序列码

元完全一致，仅存在固定的传输延迟，无错码、无抖动，验证了非相干解调链路逻辑的正确性；相干解调链路中，Costas 环可快速完成载波同步锁定，锁定后解调输出信号与原始 m 序列严格对齐，相位同步精度高，解调性能稳定，充分验证了相干解调架构的功能完整性与设计合理性。

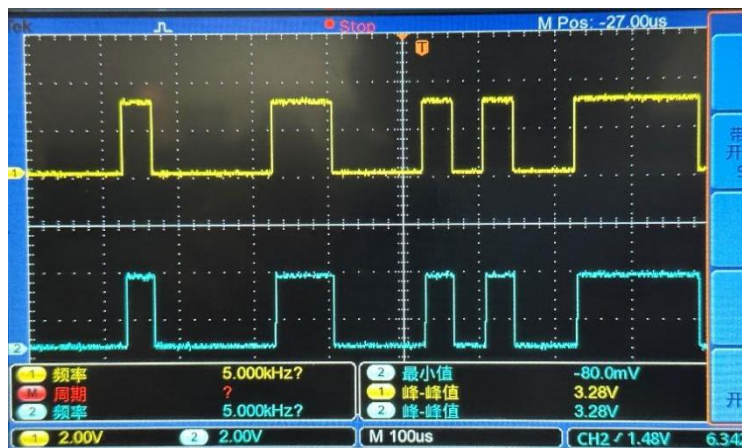


图40 整体输入输出实测图（非相干解调）

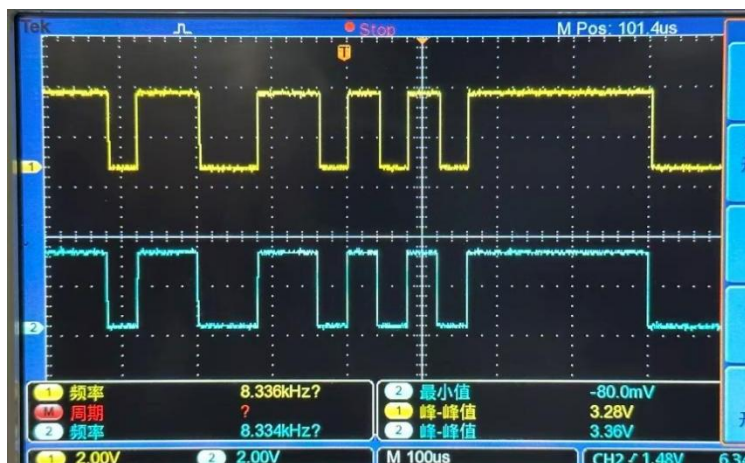


图41 整体输入输出实测图（相干解调）

由图 40 和图 41 实测测试结果显示，非相干解调模式下，示波器观测到解调输出波形与发射端原始基带 m 序列包络完全匹配，高低电平分界清晰，无波形畸变、无误码判决，可稳定还原原始基带信号；相干解调模式下，Costas 环可快速锁定接收载波的频率与相位，锁定后本地载波与接收载波实现同频同相，解调输出信号完整性更优，两种解调模式均能在实际硬件环境中可靠工作，验证了系统硬件设计的稳定性与工程实用性。

4 总结与展望

4.1 整体测试方案完成情况总结

本次课程设计基于 XSRP 软件无线电平台与 FPGA，圆满完成了 2ASK 调制解调系统的设计、编程与软硬件联调工作。在发射端，成功实现了灵活的分频控制、基于本原

多项式的伪随机序列产生以及乘法调制。在接收端，本设计不仅高标准完成了包含全波整流、滑动平均滤波和电平判决的非相干解调基础任务，更在此基础上实现了突破，成功构筑了基于极性 Costas 环的相干解调系统。特别是针对相干解调中无载波期间易导致积分器跑飞的工程痛点，创新性地引入了算术右移防溢出机制与“飞轮”保持逻辑，极大地提升了闭环系统的锁定稳定性。经示波器实测对比，各级测试点波形清晰、逻辑准确，系统成功恢复了原始基带码元，完全达到了任务书规定的各项功能与创新指标。

4.2 系统局限性分析

尽管系统实现了预期功能，但在深入调试与理论反思中，仍发现本设计存在一定的局限性：首先，在非相干解调和相干解调的判决环节，本系统采用了固定的经验门限（如包络峰值的 $1/2$ ）。在实际无线信道中，由于衰落和干扰，信号幅度会动态变化，固定门限易导致误码率上升。其次，系统采用的 128 点滑动平均滤波器虽然极大地节省了 FPGA 的乘法器资源，但其幅频响应呈现 Sinc 函数特性，旁瓣衰减有限，在抑制强邻道干扰时的性能不如基于窗函数设计的高阶 FIR 滤波器。最后，Costas 环的 PI 环路滤波器参数（ K_P 、 K_I ）为硬编码的固定常数，缺乏自适应调节能力，在应对大频偏或快变信道时，捕获时间和跟踪精度难以同时兼顾。

4.3 能力提升与未来展望

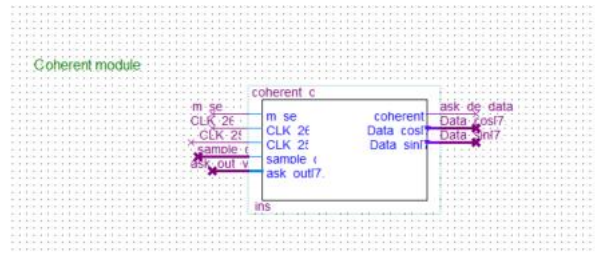
通过本次软硬件结合的通信系统设计，团队获得了全方位的能力提升。在理论层面，将《通信原理》中抽象的数学公式（如包络检波、正交混频、环路反馈）转化为了具象的数字逻辑，彻底打通了理论与工程的壁垒；在工程实践层面，深入掌握了 Verilog HDL 语言与自顶向下的 FPGA 开发流程，学会了如何在资源（除法器、乘法器）与性能之间进行权衡，并积累了丰富的 XSRP 平台使用与示波器硬件调试经验。

展望未来，通信系统的软件化与可重构化是必然趋势。针对本设计的局限性，下一步的优化方向可以考虑引入 AGC（自动增益控制）模块结合自适应门限算法以应对幅度衰落；同时，探索更为复杂的自适应环路滤波算法。此次课程设计为今后学习更高级的通信协议（如 QAM、OFDM）以及深耕现代数字信号处理领域奠定了坚实的基础。

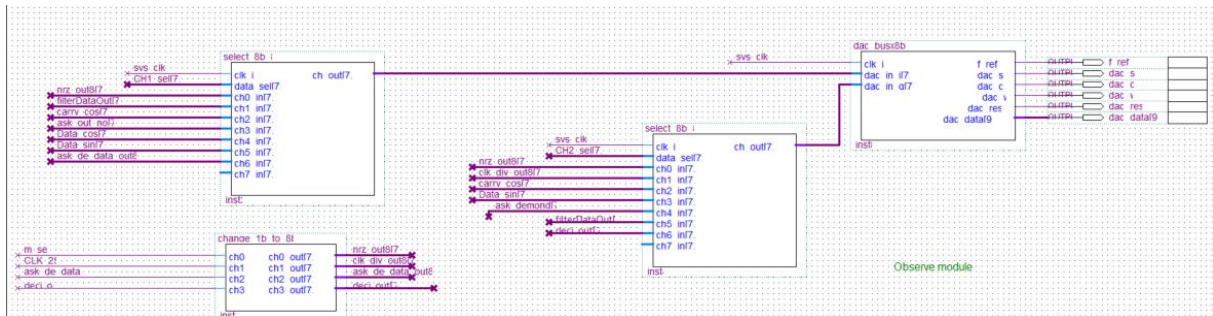
5 参考文献

- [1] 樊昌信, 曹丽娜。通信原理 [M]. 北京: 国防工业出版社, 2021.123-156
- [2] 王金明。Verilog HDL 程序设计与 FPGA 实现 [M]. 北京: 电子工业出版社, 2020.45-89
- [3] 程佩青。数字信号处理教程 [M]. 北京: 清华大学出版社, 2019.201-234
- [4] 张厥盛, 郑继禹, 万心平。锁相技术 [M]. 西安: 西安电子科技大学出版社, 2016.89-135
- [5] 杨小牛, 楼才义, 徐建良。软件无线电原理与应用 [M]. 北京: 电子工业出版社, 2018.78-112
- [6] 李辉, 张兴敢。基于 FPGA 的 2ASK 调制解调系统设计与实现 [J]. 电子测量技术, 2018,41 (12): 110-114
- [7] 王静, 刘颖。基于 DDS 的多制式数字调制系统 FPGA 实现 [J]. 微电子学与计算机, 2020,37 (5): 87-92
- [8] 陈亮, 王厚军, 戴志坚。基于 Costas 环的载波同步技术 FPGA 实现 [J]. 仪器仪表学报, 2019,40 (S1): 126-132
- [9] 刘畅。基于 FPGA 的数字调制解调系统设计与实现 [D]. 成都: 电子科技大学, 2022.23-56
- [10] 赵鑫。基于 Costas 环的 2ASK 相干解调系统 FPGA 实现 [D]. 西安: 西安电子科技大学, 2021.18-42

➤ 相干解调模块



➤ 示波器观察模块



6.2 verilog代码

➤ 分频器模块

```

module divider (
    input wire      clk_in,
    input wire [15:0] div_ratio, //分频系数
    input wire [15:0] duty_cycle, //占空比系数
    output reg      clk_out=1'b0
);
reg [15:0] cnt= 16'd0;

always @(posedge clk_in) begin
    if (div_ratio == 16'd0) begin
        cnt <= 16'd0;
        clk_out <= ~clk_out;
    end else begin
        if (cnt >= div_ratio - 16'd1) begin
            cnt <= 16'd0;
        end else begin
            cnt <= cnt + 16'd1;
        end
    end
end
    
```

```
end  
    clk_out <= (cnt < duty_cycle) ? 1'b1 : 1'b0;  
end
```

```
end
```

```
endmodule
```

➤ 载波生成模块

```
module carry_wave(  
    input CLK_26M,  
    output signed [7:0] carry_cos  
);  
    reg [8:0] rom_addr=9'd65;  
    wire [7:0]douta;  
    assign carry_cos=douta-'d127;  
  
    rom_carry inst_rom_carry(  
        .clka(CLK_26M),  
        .addra(rom_addr),  
        .douta(douta)  
    );  
    always @(posedge CLK_26M) begin  
        if (rom_addr == 9'd259) begin  
            rom_addr <= 9'd0;  
        end  
        else begin  
            rom_addr <= rom_addr + 1'b1;  
        end  
    end  
end
```

```
endmodule
```

➤ 调制波生成模块

```
module m_wave(  
    input CLK_25K,  
    output m_seq  
);
```



```
reg [5:0] m_reg=6'b111111; // 6 位移位寄存器
```

```
always @(posedge CLK_25K) begin
```

```
    m_reg <= {m_reg[4:0], m_reg[5] ^ m_reg[4]};
```

```
    // 反馈多项式  $x^6 + x^5 + 1$ 
```

```
    // 将第 6 位和第 5 位异或后输入到第 1 位
```

```
end
```

```
assign m_seq = m_reg[5]; // 取寄存器最高位作为调制信号
```

```
endmodule
```

➤ 调制模块

```
module modulator(
```

```
    input        CLK_26M,
```

```
    input        m_seq,
```

```
    input [7:0] carry_in,
```

```
    output signed [7:0] ask_out_yes,
```

```
    output signed [7:0] ask_out_no
```

```
);
```

```
wire signed [7:0] ask_out;
```

```
reg signed [7:0] ask_out_r;
```

```
assign ask_out = carry_in*m_seq;
```

```
//打拍消除竞争冒险
```

```
always @(posedge CLK_26M) begin
```

```
    ask_out_r<=ask_out;
```

```
end
```

```
assign ask_out_yes=ask_out_r;
```

```
assign ask_out_no=ask_out_r;
```

```
endmodule
```

➤ 非相干解调滤波器模块

```
module filter_LPF (
```

```
    input        clk_in,
```

```
    input [7:0] filterDataIn, // 整流后数据输入
```

```
    output [7:0] filterDataOut // 滤波后平滑输出
```

```
);
```

```

reg [128*8-1 : 0] shift_reg_bus = {(128*8){1'b0}};
reg [15:0] sum = 16'd0;
wire [7:0] oldest_data = shift_reg_bus[128*8-1 : 127*8];
wire [7:0] clean_in;
assign clean_in = (^filterDataIn == 1'bx) ? 8'd0 : filterDataIn;

always @(posedge clk_in) begin
//移位更新
//将新来的数据拼在最低位，其他数据整体向高位移动 8 位
    shift_reg_bus <= {shift_reg_bus[127*8-1 : 0], clean_in};
//累加器更新
//当前总和 = 上一次的总和 + 新进入窗口的数据 - 被挤出窗口的最老数据
    sum <= sum + clean_in - oldest_data;
end
//将 128 点的总和除以 128 以求得平均值。
//除以 128 等同于将二进制数向右移 7 位 ( $2^7 = 128$ )。
//取 sum 的 [14:7] 位作为最终输出。
assign filterDataOut = sum[14:7];
endmodule

```

➤ 非相干解调抽样判决模块

```

module decision (
    input wire      clk,
    input wire [7:0] data_in,
    output          data_out
);
reg decision_out_reg = 1'b0;

always @(posedge clk) begin
    // 门限设为包络峰值 (80) 的 1/2
    decision_out_reg <= (data_in > 8'd40) ? 1'b1 : 1'b0;
end
assign data_out = decision_out_reg;
endmodule

```

➤ 相干解调模块

```
module coherent_dem(
    input CLK_26M,
    input CLK_25K,
    input sample_clk,
    input signed [7:0]ask_out,
    output coherent_out
);
wire signed [15:0] mult_I_out;
wire signed [15:0] mult_Q_out;
wire signed [7:0] Data_cos;
wire signed [7:0] Data_sin;
wire signed [31:0] freq_adj;
// DDS 与乘法器
DDS_AD9767 inst_dds(
    .Clk(CLK_26M),
    .Data_sin(Data_sin),
    .Data_cos(Data_cos),
    .freq_adj(freq_adj)
);
mult_signed #(
    .A_WIDTH(8),
    .B_WIDTH(8)
) inst_mul_I (
    .CLK(CLK_26M),
    .A(Data_cos),
    .B(ask_out),
    .P(mult_I_out)
);

mult_signed #(
    .A_WIDTH(8),
    .B_WIDTH(8)
) inst_mul_Q (
    .CLK(CLK_26M),
```

```

        .A(Data_sin),
        .B(ask_out),
        .P(mult_Q_out)
    );
// 128 点滑动平均低通滤波
reg [128*16-1 : 0] shift_reg_I_bus= {(128*16){1'b0}};
reg [128*16-1 : 0] shift_reg_Q_bus= {(128*16){1'b0}};
reg signed [22:0] sum_I= 23'sd0;
reg signed [22:0] sum_Q= 23'sd0;

// 提取最老的一个点 (第 128 个点)
wire signed [15:0] I_oldest = shift_reg_I_bus[128*16-1 : 127*16];
wire signed [15:0] Q_oldest = shift_reg_Q_bus[128*16-1 : 127*16];

always @(posedge CLK_26M) begin
    // 移位操作: 左移 16 位, 把新数据拼在最低端
    shift_reg_I_bus <= {shift_reg_I_bus[127*16-1 : 0], mult_I_out};
    shift_reg_Q_bus <= {shift_reg_Q_bus[127*16-1 : 0], mult_Q_out};
    // 累加器更新: 加上最新值, 减去最老值
    sum_I <= sum_I + mult_I_out - I_oldest;
    sum_Q <= sum_Q + mult_Q_out - Q_oldest;
end

wire signed [15:0] I_LPF = sum_I[22:7];
wire signed [15:0] Q_LPF = sum_Q[22:7];

// 极性 Costas 环鉴相器
wire signed [15:0] phase_error_raw;
// 建立正确的负反馈闭环
assign phase_error_raw = (I_LPF[15] == 1'b0) ? (~Q_LPF + 1'b1) : Q_LPF;
wire signed [31:0] phase_error_ext = phase_error_raw;
wire signed [31:0] phase_error;
assign phase_error = phase_error_ext >>> 4;
// 环路滤波器

```

```

wire rx_flywheel_en = (I_LPF > 16'sd500); //当解调幅度足够大时，认为收到了 '1
loop_filter inst_loop_filter (
    .Clk(sample_clk),
    .code_en(rx_flywheel_en),
    .error_in(phase_error),
    .freq_adj(freq_adj)
);
// 码元判决与 coherent_out 输出
reg coherent_out_reg= 1'b0;

always @(posedge CLK_26M) begin
    // 门限设为峰值的二分之一
    coherent_out_reg <= (I_LPF > 16'sd3000) ? 1'b1 : 1'b0;
end
assign coherent_out = coherent_out_reg;
endmodule

```

➤ DDS 模块

```

module DDS_AD9767(
    input Clk,
    output signed [7:0] Data_sin,
    output signed [7:0] Data_cos,
    input signed [31:0] freq_adj
);
reg [31:0] FwordA= 32'd0;
reg [7:0] PwordA= 8'd0;

DDS_Module DDS_ModuleA(
    .Clk(Clk),
    .Fword(FwordA),
    .Pword(PwordA),
    .Data_sin(Data_sin),
    .Data_cos(Data_cos)
);
always @(posedge Clk) begin

```



```

FwordA <= $unsigned($signed({1'b0, 32'd16519105}) + freq_adj);
//将常数变为有符号数参与运算，再变为无符号数
PwordA <= 8'd32;//初始相位偏移 45 度，用于验证 Costas 环的有效性

```

```
end
```

```
//频率控制字计算公式:Fword=(目标输出频率*2^N)/工作时钟
```

```
endmodule
```

➤ DDS 频率控制、相位控制模块

```

module DDS_Module(
    input Clk,
    input [31:0] Fword,//频率控制字
    input [7:0] Pword,//相位控制字
    output wire signed [7:0] Data_sin,
    output wire signed [7:0] Data_cos
);
    wire [7:0] Data_sinr;
    wire [7:0] Data_cosr;
    // 无符号到有符号的转换
    assign Data_sin = Data_sinr - 8'd127;
    assign Data_cos = Data_cosr - 8'd127;
    // 频率控制字同步寄存器
    reg [31:0] Fword_r = 32'd0;
    always@(posedge Clk) begin
        Fword_r <= Fword;
    end
    // 相位控制字同步寄存器
    reg [7:0] Pword_r = 8'd0;
    always@(posedge Clk) begin
        Pword_r <= Pword;
    end
    // 相位累加器
    reg [31:0] Freq_ACC = 32'd0;
    always@(posedge Clk) begin
        Freq_ACC <= Fword_r + Freq_ACC;
    end
end

```

```

// 波形数据表地址
wire [7:0] Rom_Addr_sin;
wire [7:0] Rom_Addr_cos;
// 相位截断：取 32 位累加器的高 8 位
assign Rom_Addr_sin = Freq_ACC[31:24] + Pword_r;
// cos 与 sin 的 90 度偏移
assign Rom_Addr_cos = Rom_Addr_sin + 8'd64;
// ROM 例化
my_rom rom_sine(
    .clka(Clk),
    .addra(Rom_Addr_sin),
    .douta(Data_sinr)
);
my_rom rom_cos(
    .clka(Clk),
    .addra(Rom_Addr_cos),
    .douta(Data_cosr)
);
endmodule

```

➤ 环路滤波器模块

```

module loop_filter #(
    parameter ERROR_WIDTH = 32,
    parameter ADJ_WIDTH   = 32,
    parameter Kp           = 32,
    parameter Ki           = 1
)(
    input          Clk,
    input          code_en,          // 飞轮使能（来自 I_LPF 判决）
    input signed [ERROR_WIDTH-1:0] error_in,
    output reg signed [ADJ_WIDTH-1:0] freq_adj = 0
);
// 积分器
reg signed [ERROR_WIDTH+16:0] integral = 0;
// 允许最大频偏为 +-5kHz，换算成 Fword 增量大约是 +-800,000

```

```

// 这里我们把积分限幅设为大概 +-1,000,000 (20 位)
localparam signed [ERROR_WIDTH+16:0] MAX_INT = 1000000;
localparam signed [ERROR_WIDTH+16:0] MIN_INT = -1000000;
wire signed [ERROR_WIDTH+16:0] pi_out_full;
assign pi_out_full = (error_in * Kp) + (integral * Ki);

always @(posedge Clk) begin
    if(code_en) begin
        // 正常工作模式下：带有抗积分饱和的累加
        if (integral + error_in > MAX_INT) begin
            integral <= MAX_INT;    // 卡在上限
        end else if (integral + error_in < MIN_INT) begin
            integral <= MIN_INT;    // 卡在下限
        end else begin
            integral <= integral + error_in;
        end

        freq_adj <= pi_out_full >>> 4;
    end else begin
        //当没有有效信号时，积分器慢慢“泄放”归零
        // 这保证了下次信号到来时，DDS 是从标准的 100kHz 起步去追踪的，而不是处于跑飞状态
        // 通过算术右移实现缓慢衰减，比直接清零更稳定
        integral <= integral - (integral >>> 4);
        freq_adj <= 32'sd0;
    end
end
endmodule

```

➤ 乘法器模块

```

module mult_signed #(
    parameter A_WIDTH = 8,
    parameter B_WIDTH = 8
)(
    input
        CLK,

```

```

    input signed [A_WIDTH-1:0] A,
    input signed [B_WIDTH-1:0] B,
    output reg signed [A_WIDTH+B_WIDTH-1:0] P=0
);
always @(posedge CLK) begin
    P <= A * B;
end
endmodule

```

➤ 载波 ROM 模块

```

module rom_carry(
    input clka,
    input [8:0] addra,
    output reg [7:0] douta = 8'h7F
);
// 强制综合器使用 Block RAM 资源
(* rom_style="block" *) reg [7:0] rom_mem [0:259];
initial begin
    rom_mem[9'd0] = 8'h7F; rom_mem[9'd1] = 8'h82; rom_mem[9'd2] = 8'h85; rom_mem[9'd3]
    = 8'h88; rom_mem[9'd4] = 8'h8B; rom_mem[9'd5] = 8'h8E; rom_mem[9'd6] = 8'h91;
    rom_mem[9'd7] = 8'h94;
    rom_mem[9'd8] = 8'h97; rom_mem[9'd9] = 8'h9A; rom_mem[9'd10] = 8'h9D;
    rom_mem[9'd11] = 8'hA0; rom_mem[9'd12] = 8'hA3; rom_mem[9'd13] = 8'hA6;
    rom_mem[9'd14] = 8'hA9; rom_mem[9'd15] = 8'hAC;
    rom_mem[9'd16] = 8'hAF; rom_mem[9'd17] = 8'hB2; rom_mem[9'd18] = 8'hB5;
    rom_mem[9'd19] = 8'hB7; rom_mem[9'd20] = 8'hBA; rom_mem[9'd21] = 8'hBD;
    rom_mem[9'd22] = 8'hBF; rom_mem[9'd23] = 8'hC2;
    rom_mem[9'd24] = 8'hC5; rom_mem[9'd25] = 8'hC7; rom_mem[9'd26] = 8'hCA;
    rom_mem[9'd27] = 8'hCC; rom_mem[9'd28] = 8'hCF; rom_mem[9'd29] = 8'hD1;
    rom_mem[9'd30] = 8'hD3; rom_mem[9'd31] = 8'hD5;
    rom_mem[9'd32] = 8'hD8; rom_mem[9'd33] = 8'hDA; rom_mem[9'd34] = 8'hDC;
    rom_mem[9'd35] = 8'hDE; rom_mem[9'd36] = 8'hE0; rom_mem[9'd37] = 8'hE2;
    rom_mem[9'd38] = 8'hE4; rom_mem[9'd39] = 8'hE6;

```

```
rom_mem[9'd40] = 8'hE8; rom_mem[9'd41] = 8'hE9; rom_mem[9'd42] = 8'hEB;
rom_mem[9'd43] = 8'hEC; rom_mem[9'd44] = 8'hEE; rom_mem[9'd45] = 8'hEF;
rom_mem[9'd46] = 8'hF1; rom_mem[9'd47] = 8'hF2;
rom_mem[9'd48] = 8'hF3; rom_mem[9'd49] = 8'hF5; rom_mem[9'd50] = 8'hF6;
rom_mem[9'd51] = 8'hF7; rom_mem[9'd52] = 8'hF8; rom_mem[9'd53] = 8'hF9;
rom_mem[9'd54] = 8'hFA; rom_mem[9'd55] = 8'hFA;
rom_mem[9'd56] = 8'hFB; rom_mem[9'd57] = 8'hFC; rom_mem[9'd58] = 8'hFC;
rom_mem[9'd59] = 8'hFD; rom_mem[9'd60] = 8'hFD; rom_mem[9'd61] = 8'hFD;
rom_mem[9'd62] = 8'hFE; rom_mem[9'd63] = 8'hFE;
rom_mem[9'd64] = 8'hFE; rom_mem[9'd65] = 8'hFE; rom_mem[9'd66] = 8'hFE;
rom_mem[9'd67] = 8'hFE; rom_mem[9'd68] = 8'hFE; rom_mem[9'd69] = 8'hFD;
rom_mem[9'd70] = 8'hFD; rom_mem[9'd71] = 8'hFD;
rom_mem[9'd72] = 8'hFC; rom_mem[9'd73] = 8'hFC; rom_mem[9'd74] = 8'hFB;
rom_mem[9'd75] = 8'hFA; rom_mem[9'd76] = 8'hFA; rom_mem[9'd77] = 8'hF9;
rom_mem[9'd78] = 8'hF8; rom_mem[9'd79] = 8'hF7;
rom_mem[9'd80] = 8'hF6; rom_mem[9'd81] = 8'hF5; rom_mem[9'd82] = 8'hF3;
rom_mem[9'd83] = 8'hF2; rom_mem[9'd84] = 8'hF1; rom_mem[9'd85] = 8'hEF;
rom_mem[9'd86] = 8'hEE; rom_mem[9'd87] = 8'hEC;
rom_mem[9'd88] = 8'hEB; rom_mem[9'd89] = 8'hE9; rom_mem[9'd90] = 8'hE8;
rom_mem[9'd91] = 8'hE6; rom_mem[9'd92] = 8'hE4; rom_mem[9'd93] = 8'hE2;
rom_mem[9'd94] = 8'hE0; rom_mem[9'd95] = 8'hDE;
rom_mem[9'd96] = 8'hDC; rom_mem[9'd97] = 8'hDA; rom_mem[9'd98] = 8'hD8;
rom_mem[9'd99] = 8'hD5; rom_mem[9'd100] = 8'hD3; rom_mem[9'd101] = 8'hD1;
rom_mem[9'd102] = 8'hCF; rom_mem[9'd103] = 8'hCC;
rom_mem[9'd104] = 8'hCA; rom_mem[9'd105] = 8'hC7; rom_mem[9'd106] = 8'hC5;
rom_mem[9'd107] = 8'hC2; rom_mem[9'd108] = 8'hBF; rom_mem[9'd109] = 8'hBD;
rom_mem[9'd110] = 8'hBA; rom_mem[9'd111] = 8'hB7;
rom_mem[9'd112] = 8'hB5; rom_mem[9'd113] = 8'hB2; rom_mem[9'd114] = 8'hAF;
rom_mem[9'd115] = 8'hAC; rom_mem[9'd116] = 8'hA9; rom_mem[9'd117] = 8'hA6;
rom_mem[9'd118] = 8'hA3; rom_mem[9'd119] = 8'hA0;
rom_mem[9'd120] = 8'h9D; rom_mem[9'd121] = 8'h9A; rom_mem[9'd122] = 8'h97;
rom_mem[9'd123] = 8'h94; rom_mem[9'd124] = 8'h91; rom_mem[9'd125] = 8'h8E; rom_mem[9'd126]
= 8'h8B; rom_mem[9'd127] = 8'h88;
```

```
rom_mem[9'd128] = 8'h85; rom_mem[9'd129] = 8'h82; rom_mem[9'd130] = 8'h7F;  
rom_mem[9'd131] = 8'h7C; rom_mem[9'd132] = 8'h79; rom_mem[9'd133] = 8'h76; rom_mem[9'd134]  
= 8'h73; rom_mem[9'd135] = 8'h70;  
rom_mem[9'd136] = 8'h6D; rom_mem[9'd137] = 8'h6A; rom_mem[9'd138] = 8'h67;  
rom_mem[9'd139] = 8'h64; rom_mem[9'd140] = 8'h61; rom_mem[9'd141] = 8'h5E; rom_mem[9'd142]  
= 8'h5B; rom_mem[9'd143] = 8'h58;  
rom_mem[9'd144] = 8'h55; rom_mem[9'd145] = 8'h52; rom_mem[9'd146] = 8'h4F;  
rom_mem[9'd147] = 8'h4C; rom_mem[9'd148] = 8'h49; rom_mem[9'd149] = 8'h47; rom_mem[9'd150]  
= 8'h44; rom_mem[9'd151] = 8'h41;  
rom_mem[9'd152] = 8'h3F; rom_mem[9'd153] = 8'h3C; rom_mem[9'd154] = 8'h39;  
rom_mem[9'd155] = 8'h37; rom_mem[9'd156] = 8'h34; rom_mem[9'd157] = 8'h32; rom_mem[9'd158]  
= 8'h2F; rom_mem[9'd159] = 8'h2D;  
rom_mem[9'd160] = 8'h2B; rom_mem[9'd161] = 8'h29; rom_mem[9'd162] = 8'h26;  
rom_mem[9'd163] = 8'h24; rom_mem[9'd164] = 8'h22; rom_mem[9'd165] = 8'h20; rom_mem[9'd166]  
= 8'h1E; rom_mem[9'd167] = 8'h1C;  
rom_mem[9'd168] = 8'h1A; rom_mem[9'd169] = 8'h18; rom_mem[9'd170] = 8'h16;  
rom_mem[9'd171] = 8'h15; rom_mem[9'd172] = 8'h13; rom_mem[9'd173] = 8'h12; rom_mem[9'd174]  
= 8'h10; rom_mem[9'd175] = 8'h0F;  
rom_mem[9'd176] = 8'h0D; rom_mem[9'd177] = 8'h0C; rom_mem[9'd178] = 8'h0B;  
rom_mem[9'd179] = 8'h09; rom_mem[9'd180] = 8'h08; rom_mem[9'd181] = 8'h07; rom_mem[9'd182]  
= 8'h06; rom_mem[9'd183] = 8'h05;  
rom_mem[9'd184] = 8'h04; rom_mem[9'd185] = 8'h04; rom_mem[9'd186] = 8'h03;  
rom_mem[9'd187] = 8'h02; rom_mem[9'd188] = 8'h02; rom_mem[9'd189] = 8'h01; rom_mem[9'd190]  
= 8'h01; rom_mem[9'd191] = 8'h01;  
rom_mem[9'd192] = 8'h00; rom_mem[9'd193] = 8'h00; rom_mem[9'd194] = 8'h00;  
rom_mem[9'd195] = 8'h00; rom_mem[9'd196] = 8'h00; rom_mem[9'd197] = 8'h00; rom_mem[9'd198]  
= 8'h00; rom_mem[9'd199] = 8'h01;  
rom_mem[9'd200] = 8'h01; rom_mem[9'd201] = 8'h01; rom_mem[9'd202] = 8'h02;  
rom_mem[9'd203] = 8'h02; rom_mem[9'd204] = 8'h03; rom_mem[9'd205] = 8'h04; rom_mem[9'd206]  
= 8'h04; rom_mem[9'd207] = 8'h05;  
rom_mem[9'd208] = 8'h06; rom_mem[9'd209] = 8'h07; rom_mem[9'd210] = 8'h08;  
rom_mem[9'd211] = 8'h09; rom_mem[9'd212] = 8'h0B; rom_mem[9'd213] = 8'h0C; rom_mem[9'd214]  
= 8'h0D; rom_mem[9'd215] = 8'h0F;
```



```

    rom_mem[9'd216] = 8'h10; rom_mem[9'd217] = 8'h12; rom_mem[9'd218] = 8'h13;
rom_mem[9'd219] = 8'h15; rom_mem[9'd220] = 8'h16; rom_mem[9'd221] = 8'h18; rom_mem[9'd222]
= 8'h1A; rom_mem[9'd223] = 8'h1C;
    rom_mem[9'd224] = 8'h1E; rom_mem[9'd225] = 8'h20; rom_mem[9'd226] = 8'h22;
rom_mem[9'd227] = 8'h24; rom_mem[9'd228] = 8'h26; rom_mem[9'd229] = 8'h29; rom_mem[9'd230]
= 8'h2B; rom_mem[9'd231] = 8'h2D;
    rom_mem[9'd232] = 8'h2F; rom_mem[9'd233] = 8'h32; rom_mem[9'd234] = 8'h34;
rom_mem[9'd235] = 8'h37; rom_mem[9'd236] = 8'h39; rom_mem[9'd237] = 8'h3C; rom_mem[9'd238]
= 8'h3F; rom_mem[9'd239] = 8'h41;
    rom_mem[9'd240] = 8'h44; rom_mem[9'd241] = 8'h47; rom_mem[9'd242] = 8'h49;
rom_mem[9'd243] = 8'h4C; rom_mem[9'd244] = 8'h4F; rom_mem[9'd245] = 8'h52; rom_mem[9'd246]
= 8'h55; rom_mem[9'd247] = 8'h58;
    rom_mem[9'd248] = 8'h5B; rom_mem[9'd249] = 8'h5E; rom_mem[9'd250] = 8'h61;
rom_mem[9'd251] = 8'h64; rom_mem[9'd252] = 8'h67; rom_mem[9'd253] = 8'h6A; rom_mem[9'd254]
= 8'h6D; rom_mem[9'd255] = 8'h70;
    rom_mem[9'd256] = 8'h73; rom_mem[9'd257] = 8'h76; rom_mem[9'd258] = 8'h79;
rom_mem[9'd259] = 8'h7C;
end
// 时钟同步读逻辑
always @(posedge clka) begin
    douta <= rom_mem[addra];
end
Endmodule
➤ 相干解调 ROM 模块

```

```

module my_rom(//dds
    input clka,
    input [7:0] addra, // 256 个点对应 8 位地址
    output reg [7:0] douta=8'h7F
);
// 强制综合器使用 Block RAM 资源
(* rom_style="block" *) reg [7:0] rom_mem [0:255];
initial begin

```

```
rom_mem[8'h00] = 8'h7F; rom_mem[8'h01] = 8'h82; rom_mem[8'h02] = 8'h85;
rom_mem[8'h03] = 8'h88; rom_mem[8'h04] = 8'h8B; rom_mem[8'h05] = 8'h8F;
rom_mem[8'h06] = 8'h92; rom_mem[8'h07] = 8'h95;
rom_mem[8'h08] = 8'h98; rom_mem[8'h09] = 8'h9B; rom_mem[8'h0A] = 8'h9E;
rom_mem[8'h0B] = 8'hA1; rom_mem[8'h0C] = 8'hA4; rom_mem[8'h0D] = 8'hA7;
rom_mem[8'h0E] = 8'hAA; rom_mem[8'h0F] = 8'hAD;
rom_mem[8'h10] = 8'hB0; rom_mem[8'h11] = 8'hB2; rom_mem[8'h12] = 8'hB5;
rom_mem[8'h13] = 8'hB8; rom_mem[8'h14] = 8'hBB; rom_mem[8'h15] = 8'hBE;
rom_mem[8'h16] = 8'hC0; rom_mem[8'h17] = 8'hC3;
rom_mem[8'h18] = 8'hC6; rom_mem[8'h19] = 8'hC8; rom_mem[8'h1A] = 8'hCB;
rom_mem[8'h1B] = 8'hCD; rom_mem[8'h1C] = 8'hD0; rom_mem[8'h1D] = 8'hD2;
rom_mem[8'h1E] = 8'hD4; rom_mem[8'h1F] = 8'hD7;
rom_mem[8'h20] = 8'hD9; rom_mem[8'h21] = 8'hDB; rom_mem[8'h22] = 8'hDD;
rom_mem[8'h23] = 8'hDF; rom_mem[8'h24] = 8'hE1; rom_mem[8'h25] = 8'hE3;
rom_mem[8'h26] = 8'hE5; rom_mem[8'h27] = 8'hE7;
rom_mem[8'h28] = 8'hE9; rom_mem[8'h29] = 8'hEA; rom_mem[8'h2A] = 8'hEC;
rom_mem[8'h2B] = 8'hEE; rom_mem[8'h2C] = 8'hEF; rom_mem[8'h2D] = 8'hF0;
rom_mem[8'h2E] = 8'hF2; rom_mem[8'h2F] = 8'hF3;
rom_mem[8'h30] = 8'hF4; rom_mem[8'h31] = 8'hF5; rom_mem[8'h32] = 8'hF7;
rom_mem[8'h33] = 8'hF8; rom_mem[8'h34] = 8'hF9; rom_mem[8'h35] = 8'hF9;
rom_mem[8'h36] = 8'hFA; rom_mem[8'h37] = 8'hFB;
rom_mem[8'h38] = 8'hFC; rom_mem[8'h39] = 8'hFC; rom_mem[8'h3A] = 8'hFD;
rom_mem[8'h3B] = 8'hFD; rom_mem[8'h3C] = 8'hFD; rom_mem[8'h3D] = 8'hFE;
rom_mem[8'h3E] = 8'hFE; rom_mem[8'h3F] = 8'hFE;
rom_mem[8'h40] = 8'hFE; rom_mem[8'h41] = 8'hFE; rom_mem[8'h42] = 8'hFE;
rom_mem[8'h43] = 8'hFE; rom_mem[8'h44] = 8'hFD; rom_mem[8'h45] = 8'hFD;
rom_mem[8'h46] = 8'hFD; rom_mem[8'h47] = 8'hFC;
rom_mem[8'h48] = 8'hFC; rom_mem[8'h49] = 8'hFB; rom_mem[8'h4A] = 8'hFA;
rom_mem[8'h4B] = 8'hF9; rom_mem[8'h4C] = 8'hF9; rom_mem[8'h4D] = 8'hF8;
rom_mem[8'h4E] = 8'hF7; rom_mem[8'h4F] = 8'hF5;
rom_mem[8'h50] = 8'hF4; rom_mem[8'h51] = 8'hF3; rom_mem[8'h52] = 8'hF2;
rom_mem[8'h53] = 8'hF0; rom_mem[8'h54] = 8'hEF; rom_mem[8'h55] = 8'hEE;
rom_mem[8'h56] = 8'hEC; rom_mem[8'h57] = 8'hEA;
```

```
rom_mem[8'h58] = 8'hE9; rom_mem[8'h59] = 8'hE7; rom_mem[8'h5A] = 8'hE5;
rom_mem[8'h5B] = 8'hE3; rom_mem[8'h5C] = 8'hE1; rom_mem[8'h5D] = 8'hDF;
rom_mem[8'h5E] = 8'hDD; rom_mem[8'h5F] = 8'hDB;
rom_mem[8'h60] = 8'hD9; rom_mem[8'h61] = 8'hD7; rom_mem[8'h62] = 8'hD4;
rom_mem[8'h63] = 8'hD2; rom_mem[8'h64] = 8'hD0; rom_mem[8'h65] = 8'hCD;
rom_mem[8'h66] = 8'hCB; rom_mem[8'h67] = 8'hC8;
rom_mem[8'h68] = 8'hC6; rom_mem[8'h69] = 8'hC3; rom_mem[8'h6A] = 8'hC0;
rom_mem[8'h6B] = 8'hBE; rom_mem[8'h6C] = 8'hBB; rom_mem[8'h6D] = 8'hB8;
rom_mem[8'h6E] = 8'hB5; rom_mem[8'h6F] = 8'hB2;
rom_mem[8'h70] = 8'hB0; rom_mem[8'h71] = 8'hAD; rom_mem[8'h72] = 8'hAA;
rom_mem[8'h73] = 8'hA7; rom_mem[8'h74] = 8'hA4; rom_mem[8'h75] = 8'hA1;
rom_mem[8'h76] = 8'h9E; rom_mem[8'h77] = 8'h9B;
rom_mem[8'h78] = 8'h98; rom_mem[8'h79] = 8'h95; rom_mem[8'h7A] = 8'h92;
rom_mem[8'h7B] = 8'h8F; rom_mem[8'h7C] = 8'h8B; rom_mem[8'h7D] = 8'h88;
rom_mem[8'h7E] = 8'h85; rom_mem[8'h7F] = 8'h82;
rom_mem[8'h80] = 8'h7F; rom_mem[8'h81] = 8'h7C; rom_mem[8'h82] = 8'h79;
rom_mem[8'h83] = 8'h76; rom_mem[8'h84] = 8'h73; rom_mem[8'h85] = 8'h6F;
rom_mem[8'h86] = 8'h6C; rom_mem[8'h87] = 8'h69;
rom_mem[8'h88] = 8'h66; rom_mem[8'h89] = 8'h63; rom_mem[8'h8A] = 8'h60;
rom_mem[8'h8B] = 8'h5D; rom_mem[8'h8C] = 8'h5A; rom_mem[8'h8D] = 8'h57;
rom_mem[8'h8E] = 8'h54; rom_mem[8'h8F] = 8'h51;
rom_mem[8'h90] = 8'h4E; rom_mem[8'h91] = 8'h4C; rom_mem[8'h92] = 8'h49;
rom_mem[8'h93] = 8'h46; rom_mem[8'h94] = 8'h43; rom_mem[8'h95] = 8'h40;
rom_mem[8'h96] = 8'h3E; rom_mem[8'h97] = 8'h3B;
rom_mem[8'h98] = 8'h38; rom_mem[8'h99] = 8'h36; rom_mem[8'h9A] = 8'h33;
rom_mem[8'h9B] = 8'h31; rom_mem[8'h9C] = 8'h2E; rom_mem[8'h9D] = 8'h2C;
rom_mem[8'h9E] = 8'h2A; rom_mem[8'h9F] = 8'h27;
rom_mem[8'hA0] = 8'h25; rom_mem[8'hA1] = 8'h23; rom_mem[8'hA2] = 8'h21;
rom_mem[8'hA3] = 8'h1F; rom_mem[8'hA4] = 8'h1D; rom_mem[8'hA5] = 8'h1B;
rom_mem[8'hA6] = 8'h19; rom_mem[8'hA7] = 8'h17;
rom_mem[8'hA8] = 8'h15; rom_mem[8'hA9] = 8'h14; rom_mem[8'hAA] = 8'h12;
rom_mem[8'hAB] = 8'h10; rom_mem[8'hAC] = 8'h0F; rom_mem[8'hAD] = 8'h0E;
rom_mem[8'hAE] = 8'h0C; rom_mem[8'hAF] = 8'h0B;
```

```

rom_mem[8'hB0] = 8'h0A; rom_mem[8'hB1] = 8'h09; rom_mem[8'hB2] = 8'h07;
rom_mem[8'hB3] = 8'h06; rom_mem[8'hB4] = 8'h05; rom_mem[8'hB5] = 8'h05;
rom_mem[8'hB6] = 8'h04; rom_mem[8'hB7] = 8'h03;
rom_mem[8'hB8] = 8'h02; rom_mem[8'hB9] = 8'h02; rom_mem[8'hBA] = 8'h01;
rom_mem[8'hBB] = 8'h01; rom_mem[8'hBC] = 8'h01; rom_mem[8'hBD] = 8'h00;
rom_mem[8'hBE] = 8'h00; rom_mem[8'hBF] = 8'h00;
rom_mem[8'hC0] = 8'h00; rom_mem[8'hC1] = 8'h00; rom_mem[8'hC2] = 8'h00;
rom_mem[8'hC3] = 8'h00; rom_mem[8'hC4] = 8'h01; rom_mem[8'hC5] = 8'h01;
rom_mem[8'hC6] = 8'h01; rom_mem[8'hC7] = 8'h02;
rom_mem[8'hC8] = 8'h02; rom_mem[8'hC9] = 8'h03; rom_mem[8'hCA] = 8'h04;
rom_mem[8'hCB] = 8'h05; rom_mem[8'hCC] = 8'h05; rom_mem[8'hCD] = 8'h06;
rom_mem[8'hCE] = 8'h07; rom_mem[8'hCF] = 8'h09;
rom_mem[8'hD0] = 8'h0A; rom_mem[8'hD1] = 8'h0B; rom_mem[8'hD2] = 8'h0C;
rom_mem[8'hD3] = 8'h0E; rom_mem[8'hD4] = 8'h0F; rom_mem[8'hD5] = 8'h10;
rom_mem[8'hD6] = 8'h12; rom_mem[8'hD7] = 8'h14;
rom_mem[8'hD8] = 8'h15; rom_mem[8'hD9] = 8'h17; rom_mem[8'hDA] = 8'h19;
rom_mem[8'hDB] = 8'h1B; rom_mem[8'hDC] = 8'h1D; rom_mem[8'hDD] = 8'h1F;
rom_mem[8'hDE] = 8'h21; rom_mem[8'hDF] = 8'h23;
rom_mem[8'hE0] = 8'h25; rom_mem[8'hE1] = 8'h27; rom_mem[8'hE2] = 8'h2A;
rom_mem[8'hE3] = 8'h2C; rom_mem[8'hE4] = 8'h2E; rom_mem[8'hE5] = 8'h31;
rom_mem[8'hE6] = 8'h33; rom_mem[8'hE7] = 8'h36;
rom_mem[8'hE8] = 8'h38; rom_mem[8'hE9] = 8'h3B; rom_mem[8'hEA] = 8'h3E;
rom_mem[8'hEB] = 8'h40; rom_mem[8'hEC] = 8'h43; rom_mem[8'hED] = 8'h46;
rom_mem[8'hEE] = 8'h49; rom_mem[8'hEF] = 8'h4C;
rom_mem[8'hF0] = 8'h4E; rom_mem[8'hF1] = 8'h51; rom_mem[8'hF2] = 8'h54;
rom_mem[8'hF3] = 8'h57; rom_mem[8'hF4] = 8'h5A; rom_mem[8'hF5] = 8'h5D;
rom_mem[8'hF6] = 8'h60; rom_mem[8'hF7] = 8'h63;
rom_mem[8'hF8] = 8'h66; rom_mem[8'hF9] = 8'h69; rom_mem[8'hFA] = 8'h6C;
end
// 时钟同步读逻辑
always @(posedge clka) begin
    douta <= rom_mem[addra];
end
endmodule

```